

if consteval

Document #: P1938R1
Date: 2020-02-17
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Richard Smith
<richard@metafoo.co.uk>
Andrew Sutton
<asutton@lock3software.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

[1 Revision history](#)

[2 Introduction](#)

[3 Problems with Status Quo](#)

[3.1 Interaction between constexpr and consteval](#)

[3.2 The if constexpr \(std::is_constant_evaluated\(\)\) problem](#)

[3.2.1 Compiler warnings](#)

[4 Proposal](#)

[4.1 Negated form](#)

[4.2 Conditioned Form](#)

[4.3 Deprecating std::is_constant_evaluated\(\)](#)

[4.4 Examples](#)

[5 History](#)

[6 Wording](#)

[6.1 Feature test macro](#)

[7 Acknowledgments](#)

[8 References](#)

§ 1 Revision history

R0 [[P1938R0](#)] of this paper initially contained only a positive form: `if consteval`. This paper additionally adds a negated form, `if not consteval`.

§ 2 Introduction

Despite this paper missing both our respective NB comment deadlines and the mailing deadline, we still believe it provides a significant enough improvement to the status quo that it should be considered for C++20.

C++20 will have several new features to aid programmers in writing code during constant evaluation. Two of these are `std::is_constant_evaluated()` [P0595R2] and `constexpr` [P1073R3], both adopted in San Diego 2018. `constexpr` is for functions that can only be invoked during constant evaluation. `is_constant_evaluated()` is a magic library function to check if the current evaluation is constant evaluation to provide, for instance, a valid implementation of an algorithm for constant evaluation time and a better implementation for runtime.

However, despite being adopted at the same meeting, these features interact poorly with each other and have other issues that make them ripe for confusion.

§ 3 Problems with Status Quo

There are two problems this paper wishes to address.

§ 3.1 Interaction between `constexpr` and `constexpr`

The first problem is the interplay between this magic library function and the new `constexpr`. Consider the example:

```
constexpr int f(int i) { return i; }

constexpr int g(int i) {
    if (std::is_constant_evaluated()) {
        return f(i) + 1; // <==
    } else {
        return 42;
    }
}

constexpr int h(int i) {
    return f(i) + 1;
}
```

The function `h` here is basically a lifted, constant-evaluation-only version of the function `g`. At constant evaluation time, they do the same thing, except that during runtime, you cannot call `h`, and `g` has this extra path. Maybe this code started with just `h` and someone decided a runtime version would also be useful and turned it into `g`.

Unfortunately, `h` is well-formed while `g` is ill-formed. You cannot make that call to `f` (that is ominously marked with an arrow) in that location. Even though that call will *only* happen during constant evaluation, that's still not enough.

With specific terms, the call to `f()` inside of `g()` is an *immediate invocation* and needs to be a constant expression and it is not. Whereas the call to `f()` inside of `h()` is *not* considered an *immediate invocation*

because it is in an *immediate function context* (i.e. it's invoked from another immediate function), so it has a weaker set of restrictions that it needs to follow.

In other words, this kind of construction of conditionally invoking a `constexpr` function from a `constexpr` function just Does Not Work (modulo the really trivial cases - one could call `f(42)` for instance, just never `f(i)`).

We find this lack of composability of features to be problematic and think it can be improved.

§ 3.2 The `if constexpr (std::is_constant_evaluated())` problem

The second problem is specific to `is_constant_evaluated`. Once you learn what this magic function is for, the obvious usage of it is:

```
constexpr size_t strlen(char const* s) {
    if constexpr (std::is_constant_evaluated()) {
        for (const char *p = s; ; ++p) {
            if (*p == '\0') {
                return static_cast<std::size_t>(p - s);
            }
        }
    } else {
        __asm__("SSE 4.2 insanity");
    }
}
```

This example, inspired by [P1045R0], has a bug: it uses `if constexpr` to check the conditional `is_constant_evaluated()` rather than a simple `if`. You have to really deeply understand a lot about how constant evaluation works in C++ to understand that this is in fact not only *not* “obviously correct” but is in fact “obviously incorrect,” for some definition of obvious. This is such a likely source of error that Barry submitted bugs to both [gcc](#) and [clang](#) to encourage the compilers to warn on such improper usage. gcc 10.1 will provide a warning for the [simple case](#):

```
<source>: In function 'constexpr int f(int)':
<source>:4:45: warning: 'std::is_constant_evaluated' always evaluates to true in 'if
      constexpr' [-Wtautological-compare]
   4 |         if constexpr (std::is_constant_evaluated()) {
     |                               ~~~~~^~
```

But then people have to understand why this is a warning, and what this even means. Nevertheless, a compiler warning is substantially better than silently wrong code, but it is problematic to have an API in which many users are drawn to a usage that is tautologically incorrect.

§ 3.2.1 Compiler warnings

When R0 of this paper was presented in Belfast, the implementers assured that all the compilers would properly warn on all tautological uses of `std::is_constant_evaluated()` - both in the always-`true` and always-`false` cases.

As of this writing, for instance, EDG warns on all of the following:

```
constexpr int f1() {
    if constexpr (!std::is_constant_evaluated() && sizeof(int) == 4) { // warning:
        always true
        return 0;
    }
    if (std::is_constant_evaluated()) {
        return 42;
    } else {
        if constexpr (std::is_constant_evaluated()) { // warning: always true
            return 0;
        }
    }
    return 7;
}

constexpr int f2() {
    if (std::is_constant_evaluated() && f1()) { // warning: always true
        return 42;
    }
    return 7;
}

int f3() {
    if (std::is_constant_evaluated() && f1()) { // warning: always false
        return 42;
    }
    return 7;
}
```

We expect the other compilers to follow suit.

§ 4 Proposal

We propose a new form of `if` statement which is spelled:

```
if constexpr { }
```

The braces (in both the `if` and the optional `else`) are mandatory and there is no condition. If evaluation of this statement occurs during constant evaluation, the first substatement is executed. Otherwise, the second substatement (if there is one) is executed.

This behaves exactly as today's:

```
if (std::is_constant_evaluated()) { }
```

except with three differences:

1. No header include is necessary.

2. The syntax is different, which completely sidesteps the confusion over the proper way to check if we're in constant evaluation. You simply cannot misuse the syntax.
3. We can use `if consteval` to allow invoking immediate functions.

To explain the last point a bit more, the current language rules allow you to invoke a `constexpr` function from inside of another `constexpr` function ([\[expr.const\]/12](#)) - we can do this by construction:

An expression or conversion is in an *immediate function context* if it is potentially evaluated and its innermost non-block scope is a function parameter scope of an immediate function.

An expression or conversion is an *immediate invocation* if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

By extending the term *immediate function context* to also include an `if consteval` block, we can allow the second example to work:

```
constexpr int f(int i) { return i; }

constexpr int g(int i) {
    if consteval {
        return f(i) + 1; // ok: immediate function context
    } else {
        return 42;
    }
}

constexpr int h(int i) {
    return f(i) + 1; // ok: immediate function context
}
```

Additionally, such a feature would allow for an easy implementation of the original `std::is_constant_evaluated()`:

```
constexpr bool is_constant_evaluated() {
    if consteval {
        return true;
    } else {
        return false;
    }
}
```

Although this paper does not suggest removing the library function.

§ 4.1 Negated form

Many people have expressed the view that a negated form is also useful. That form is also proposed here, spelled:

```
if not consteval { }
```

or

```
if ! consteval { }
```

With the semantics that the first substatement is executed if the context is *not* manifestly constant evaluated, otherwise the second substatement (if any) is executed.

§ 4.2 Conditioned Form

As proposed, this new form of `if` does not have a condition - unlike the other two we already have. While there are certainly cases where an added condition would be useful, this paper is deliberately not including such a thing. The vast majority of uses are expected to be just of the `if consteval` or `if not consteval` form and we do not want to clutter future design space in this area.

There are currently two uses in libstdc++ that are of the form `if (is_constant_evaluated() && cond)`. [One example](#):

```
if (std::is_constant_evaluated() && __n < 0)
    throw "attempt to decrement a non-bidirectional iterator";
```

This usage is perfectly fine and doesn't necessary need special support from this proposal. Or it could also be written as:

```
if consteval {
    if (__n < 0) throw "attempt to decrement a non-bidirectional iterator";
}
```

Or factored into a function like:

```
if consteval {
    consteval_assert(__n >= 0,
        "attempt to decrement a non-bidirectional iterator");
}
```

Either way, the condition form doesn't feel strongly motivated except for consistency with `if` and `if constexpr`.

§ 4.3 Deprecating `std::is_constant_evaluated()`

One of the questions that comes up regularly in discussing this paper is: if we had `if consteval`, we do we even need `std::is_constant_evaluated()`, and can we just deprecate it?

This paper proposes no such deprecation. The reason is that this function is actually still occasionally useful (as in the previous section). If the standard library does not provide it, users will write their own. We're not concerned about the implementation difficulty of it - the users that need this will definitely be able to write it correctly - but we are concerned with a proliferation of exactly this function. The advantage of having the one `std::is_constant_evaluated()` is both that it becomes actually teachable and also that it becomes warnable: the warnings discussed can happen only because we know what this name means. Maybe it's still possible to warn on `if constexpr (your::is_constant_evaluated())` but that's a much harder problem.

And note that libstdc++ already has [some uses](#) that do require the function form.

§ 4.4 Examples

Here are a few examples from libstdc++. Today, they're implemented uses a builtin function, and how they would look with `if constexpr`. It's not a big difference, just spelling.

From libstdc++	Proposed
<pre>if (!__builtin_is_constant_evaluated()) __glibcxx_assert(__shift_exponent != numeric_limits<Tp>::digits);</pre>	<pre>if not constexpr { __glibcxx_assert(__shift_exponent != numeric_limits<Tp>::digits); }</pre>
<pre>if (__builtin_is_constant_evaluated()) return __x < __y; return (__UINTPTR_TYPE__)__x > (__UINTPTR_TYPE__)__y;</pre>	<pre>if constexpr { return __x < __y; } else { return (__UINTPTR_TYPE__)__x > (__UINTPTR_TYPE__)__y; }</pre>

As of this writing, libstdc++ has 23 uses that could be replaced by `if constexpr`, 2 that could be replaced by `if not constexpr`, and 2 that require an extra condition on the comparison.

§ 5 History

The initial revision of the `std::is_constant_evaluated()` proposal [P0595R0] was actually targeted as a language feature rather than a library feature. The original spelling was `if (constexpr())`. The paper was presented in Kona 2017 and was received very favorably in the form it was presented (17-4). The poll to consider a magic library alternative was only marginally more preferred (17-3). We believe that in the two years since these polls were taken, having a dedicated language feature with an impossible-to-misuse API, that can coexist with the rest of the constant ecosystem, is the right direction.

§ 6 Wording

Extend the definition of immediate function context in 7.7 [expr.const] (and use bullet points):

An expression or conversion is in an *immediate function context* if it is potentially evaluated and **either**:

- (12.1) — its innermost non-block scope is a function parameter scope of an immediate function;
or
- (12.2) — it appears in the first *compound-statement* of a `constexpr` if statement ([stmt.if]) of the form `if constexpr` or the second *compound-statement* (if any) of a `constexpr` if

statement of the form `if ! consteval`.

Change 8.5 [stmt.select] to add the new grammar:

```
selection-statement:
    if constexpropt ( init-statementopt condition ) statement
    if constexpropt ( init-statementopt condition ) statement else statement
+   if !opt consteval compound-statement
+   if !opt consteval compound-statement else compound-statement
    switch ( init-statementopt condition ) statement
```

Add a new clause to 8.5.1 [stmt.if]

- a An `if` statement is of the form `if consteval` or `if ! consteval` is called a *consteval if* statement.
- b If the `if` statement is of the form `if consteval` and evaluation occurs in a context that is manifestly constant-evaluated ([expr.const]), the first substatement is executed and is an immediate function context ([expr.const]). Otherwise, if the `else` part of the selection statement is present, then the second substatement is executed. A `case` or `default` label appearing within such an `if` statement shall be associated with a `switch` statement within the same `if` statement. A label declared in a substatement of an `consteval if` statement shall only be referred to by a statement in the same substatement.
- c A `consteval if` statement of the form `if ! consteval compound-statement` is equivalent to
`if consteval { } else compound-statement`

A `consteval if` statement of the form `if ! consteval compound-statement1 else compound-statement2` is equivalent to
`if consteval compound-statement2 else compound-statement1`

Change 20.15.10 [meta.const.eval] to use this new functionality:

```
constexpr bool is_constant_evaluated() noexcept;
```

- 1 ~~Returns: true if and only if evaluation of the call occurs within the evaluation of an expression or conversion that is manifestly constant-evaluated ([expr.const]).~~

- 1 Effects: Equivalent to:

```
if consteval {
    return true;
} else {
    return false;
}
```

§ 6.1 Feature test macro

Add the macro `__cpp_if_consteval`.

§ 7 Acknowledgments

Thank you to David Stone and Tim Song for working through these examples.

§ 8 References

[P0595R0] Daveed Vandevoorde. 2017. The “constexpr” Operator.

<https://wg21.link/p0595r0>

[P0595R2] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018. `std::is_constant_evaluated`.

<https://wg21.link/p0595r2>

[P1045R0] David Stone. 2018. constexpr Function Parameters.

<https://wg21.link/p1045r0>

[P1073R3] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018. Immediate functions.

<https://wg21.link/p1073r3>

[P1938R0] Barry Revzin, Daveed Vandevoorde, Richard Smith. 2019. `if consteval`.

<https://wg21.link/p1938r0>